Character-Level N-gram Language Models for Hangman

Philip Umeadi
MAT 490: Mathematical Foundations of Machine Learning
Syracuse University
pumeadi@@syr.edu

Abstract

We frame Hangman as a character-level prediction problem, where the player must identify a hidden word one letter at a time with no initial context beyond its length. We construct unigram, bigram, and trigram language models and combine them via linear interpolation, with mixture weights optimized on held-out data by minimizing perplexity. On a 370,000-word English corpus, the optimizer assigns full weight to the trigram component, achieving an average of 8.2 incorrect guesses per word; when training data is scarce, the weights shift toward lower-order models, confirming that the method adapts to corpus coverage. Extending to 4-gram and 5-gram models degrades performance due to context sparsity. We also validate perplexity as an intrinsic evaluation metric for this task by showing that it correlates positively with per-word mistake count.

1 Introduction

Hangman is a guessing game where the player identifies a hidden word by proposing one letter at a time. Correct guesses reveal all instances of that letter; incorrect guesses incur penalties. The only initial information is the word's length. The aim of this project is to apply ideas developed for large language models in a more tractable setting, by training and evaluating character-level n-gram models on this task.

We treat Hangman as a character-level prediction problem. At each turn, the partially revealed word provides a sequence of known and unknown characters. For every possible next guess, we estimate the probability that a candidate letter fills one or more blank positions, conditioned on the surrounding context. This approach aligns directly with the n-gram language modeling framework described by Jurafsky and Martin [1], where the probability of the next character depends on the preceding $n-1$ characters.

The primary challenge is selecting the optimal model order. A unigram model considers only overall letter frequencies and ignores context. A bigram uses one preceding character; a trigram uses two. Higher-order models provide more context but face data sparsity because most four- or five-character contexts are absent from the training corpus, making their probability estimates unreliable. This balance between model complexity and reliability reflects the bias-variance trade-off discussed in James et al. [2].

Rather than committing to a single n-gram order, we combine unigram, bigram, and trigram models using linear interpolation [1], where the weights $(\lambda_1, \lambda_2, \lambda_3)$ sum to one and are optimized on held-out data by minimizing perplexity. With more training data, higher-order models receive higher weights; with less, lower-order models compensate for sparse contexts.

We evaluate our system in two ways. Intrinsic evaluation uses perplexity, which assesses how well the model predicts each character; lower perplexity means better predictions. Extrinsic evaluation measures the average number of incorrect guesses per word in simulated Hangman games. We show that these two scores are correlated, so perplexity serves as a useful proxy for task performance. Our experiments show: (i) interpolation weights shift toward lower-order models as training data decreases, confirming that the method adapts automatically to data availability, (ii) trigram is the best single model order, and (iii) extending to 4-gram and 5-gram models degrades performance due to context sparsity.

2 Model Construction

This section develops the mathematical framework behind the n-gram language model and describes the design decisions that follow from it. We begin with the n-gram approximation and maximum likelihood estimation, then address the sparsity problem through smoothing, and conclude with the evaluation metrics and solver strategy.

2.1 N-gram Language Models

The probability of observing a specific word, viewed as a sequence of characters $c_1, c_2, \ldots, c_N$, decomposes by the chain rule of probability into

$$P(c_1, c_2, \ldots, c_N) = \prod_{i=1}^{N} P(c_i \mid c_1, \ldots, c_{i-1}) \tag{1}$$

Each factor conditions on the entire preceding history, which becomes intractable to estimate because the number of possible histories grows exponentially with word length. The n-gram approximation addresses this by assuming that each character depends only on the preceding $n-1$ characters:

$$P(c_i \mid c_1, \ldots, c_{i-1}) \approx P(c_i \mid c_{i-n+1}, \ldots, c_{i-1}) \tag{2}$$

This reduces the conditioning context to a fixed-length window, making the probabilities estimable from corpus counts.

To apply this at word boundaries, each word is padded with $n-1$ start tokens and a single end token. For a trigram model, the word "hello" becomes `[<s>, <s>, h, e, l, l, o, </s>]`. This ensures that the model can estimate probabilities for the first characters of a word, where the natural context is shorter than the required $n-1$ characters. N-grams are then extracted by sliding a window of size n across each padded word.

2.2 Maximum Likelihood Estimation

The maximum likelihood estimate for an n-gram probability is the ratio of counts:

$$P_{\text{MLE}}(c_i \mid c_{i-n+1:i-1}) = \frac{C(c_{i-n+1:i})}{C(c_{i-n+1:i-1})} \tag{3}$$

where $C(\cdot)$ denotes the number of occurrences of the given sequence in the training data. We train separate models for orders one through three. The unigram model uses an empty context and captures only marginal character frequencies. The bigram conditions on one preceding character, and the trigram on two.

This estimator is simple but assigns zero probability to any n-gram absent from the training data. Valid sequences in the test set that were unseen during training receive probability zero, which is both unrealistic and, as we will see in Section 2.4, catastrophic for perplexity.

2.3 Smoothing via Linear Interpolation

One standard remedy for zero counts is Laplace smoothing, which adds a small constant k to every n-gram count before normalization:

$$P_{\text{Laplace}}(c_i \mid c_{i-n+1:i-1}) = \frac{C(c_{i-n+1:i}) + k}{C(c_{i-n+1:i-1}) + k|V|} \tag{4}$$

where $|V|$ is the vocabulary size. This guarantees nonzero probabilities but treats all unseen events as equally likely. More importantly, the method cannot fall back on the statistics of a shorter context. When an entire context has never been observed, adding k to a zero count still produces a poor estimate. This limitation motivates interpolation.

Linear interpolation combines the predictions of multiple n-gram orders into a single estimate:

$$P_{\text{interp}}(c_i \mid c_{i-n+1:i-1}) = \sum_{j=1}^{n} \lambda_j \, P_j(c_i \mid c_{i-j+1:i-1}) \tag{5}$$

subject to: $\sum_{j=1}^{n} \lambda_j = 1, \quad \lambda_j \geq 0 \ \ \forall j.$

When a higher-order context is well attested, its contribution dominates. When that context is rare or unseen, lower-order components serve as a fallback. The weights $\lambda_1, \ldots, \lambda_n$ are set by partitioning the training data into a primary training set and a held-out set. We train the n-gram models on the primary set and choose the weights that minimize perplexity on the held-out set via grid search over all $(\lambda_1, \lambda_2, \lambda_3)$ triples at a step size of 0.1, subject to the constraint that they sum to one.

This procedure has a useful self-regulating property. When the corpus is large and higher-order contexts are well covered, the optimizer assigns most weight to the highest-order model. When the corpus is small and many high-order contexts are unattested, the optimizer distributes weight across all orders. We observe both behaviors in our experiments (Section 3.5).

2.4 Perplexity

Perplexity is the standard intrinsic evaluation metric for language models. For a test sequence of N characters, it is defined as

$$\text{PP} = 2^{\,-\frac{1}{N}\sum_{i=1}^{N} \log_2 P(c_i | c_{i-n+1:i-1})} \tag{6}$$

The exponent is the negative average log-probability per character, so perplexity can be interpreted as the effective number of equally likely characters the model is choosing among at each position. A perfect model achieves a perplexity of one. A model that guesses uniformly over 27 characters (26 letters plus the end token) achieves a perplexity of 27. Any useful language model falls between these extremes.

Perplexity as an intrinsic metric evaluates the model's predictions without reference to application. In our setting, we consider the average number of incorrect guesses in Hangman our extrinsic metric. Whether these two measures agree is not obvious. We investigate the relationship empirically in Section 3.3.

2.5 Hangman Solver

The solver translates the model's probability estimates into guesses. Given a partially revealed word state, the trained models, the optimized weights, and the set of previously guessed letters, the solver pads the state with boundary tokens and iterates over each blank position. For each blank, it extracts the n-gram context from the surrounding characters and computes the interpolated probability for every candidate letter not yet guessed. These probabilities are summed across all blank positions, and the solver returns the letter with the highest aggregate score.

This summing strategy suits Hangman because a single guess reveals all occurrences of the guessed letter. A letter with moderate probability at several blanks is often a better guess than a letter with high probability at only one, since a correct guess at multiple positions reveals more of the word per turn.

3 Experiments and Results

The training corpus consists of 370,103 English words drawn from a standard dictionary file [3], one word per line. We split this corpus into a training set (95%) and a test set (5%), then further partition the training set into a primary training portion (90%) and a held-out set (10%) for lambda optimization. All splits use a fixed random seed for reproducibility.

3.1 Intrinsic Evaluation: Perplexity

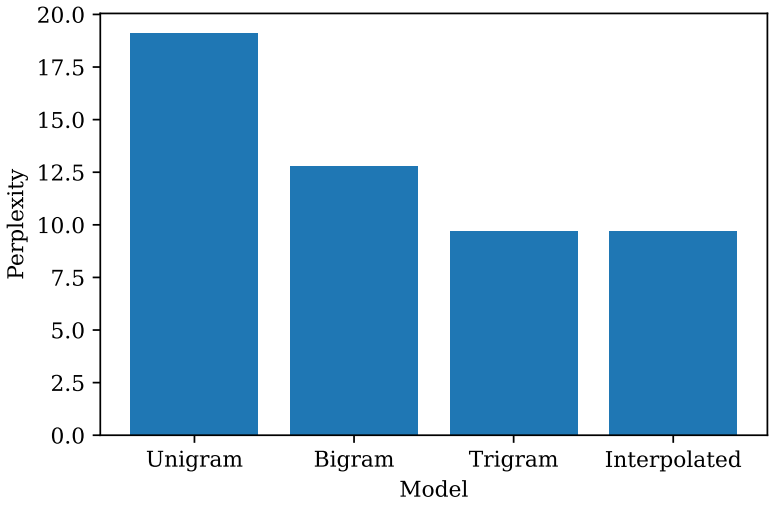

Figure 1: Perplexity decreases as model complexity increases from unigram to trigram. The interpolated model matches the trigram, assigning $\lambda_3 = 1.0$.

We computed perplexity on the test set for each n-gram order in isolation and for the interpolated model. Results are shown in Figure 1. Perplexity drops substantially from unigram (19.1) to bigram (12.8), reflecting the value of even a single character of context. The trigram improves further to 9.7, though with diminishing returns. The optimizer assigns $\lambda_3 = 1.0$, placing all weight on the trigram. We explain why in Section 3.4.

3.2 Extrinsic Evaluation: Average Mistakes

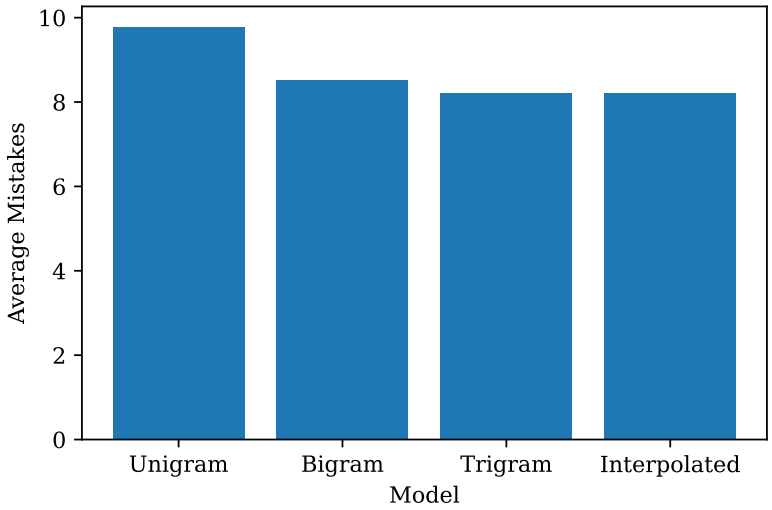

Figure 2: Average mistakes per word decrease with model order, mirroring the perplexity trend. The interpolated model matches trigram performance.

Figure 2 reports the average number of incorrect guesses per word under the same four configurations. The ordering mirrors the perplexity results: unigram (9.8), bigram (8.5), trigram (8.2), interpolated (8.2). That both metrics agree in ranking is not guaranteed but the consistency suggests that the solver translates model quality into gameplay performance effectively.

3.3 Perplexity–Mistakes Correlation

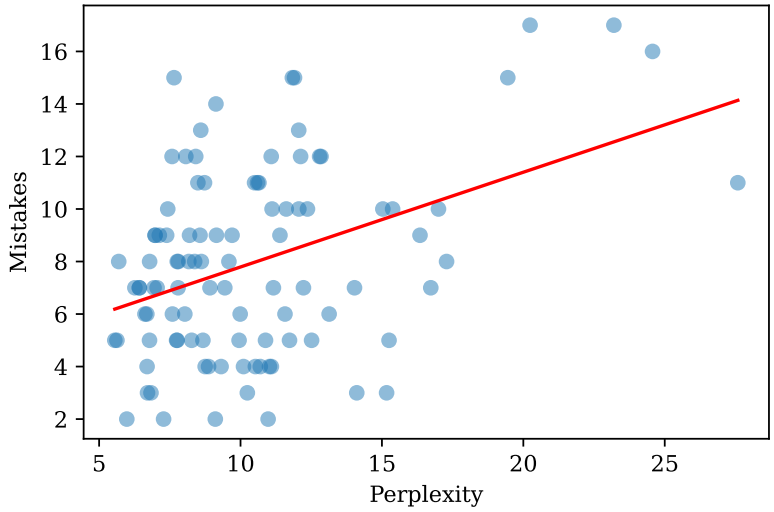

Figure 3: Per-word perplexity correlates positively with mistakes. The trend line confirms that perplexity is a useful, if noisy, predictor of individual word difficulty.

To test whether the agreement holds at the individual word level, we computed per-word perplexity and mistake counts for 100 test words. Figure 3 shows a positive trend. The correlation is noisy because short words can incur many mistakes despite low perplexity, and long words with moderate perplexity may be solved efficiently, but the overall direction confirms that perplexity is a useful predictor of word-level difficulty.

3.4 Lambda Weight Analysis

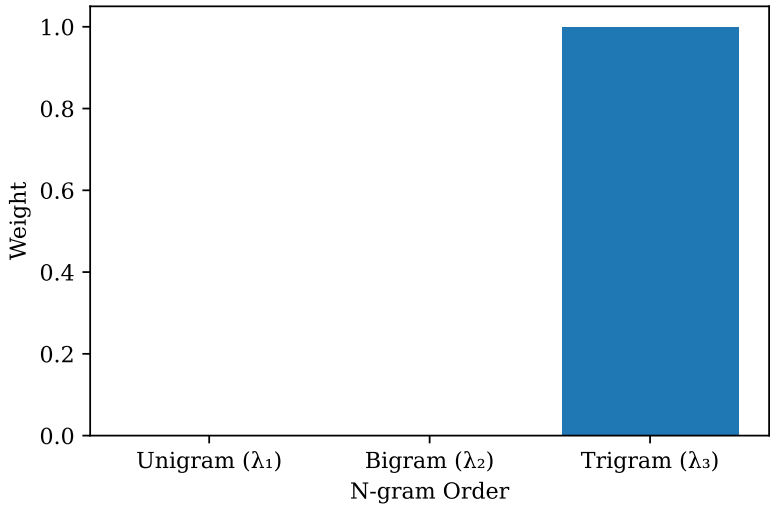

Figure 4: Optimized interpolation weights assign all mass to the trigram model when trained on the full corpus.

Figure 4 displays the optimized interpolation weights trained on the full corpus. The trigram receives all of the weight. With over 300,000 training words, most trigram contexts are well attested, and backing off to shorter contexts offers no benefit. Interpolation correctly identifies this.

3.5 Training Size Sensitivity

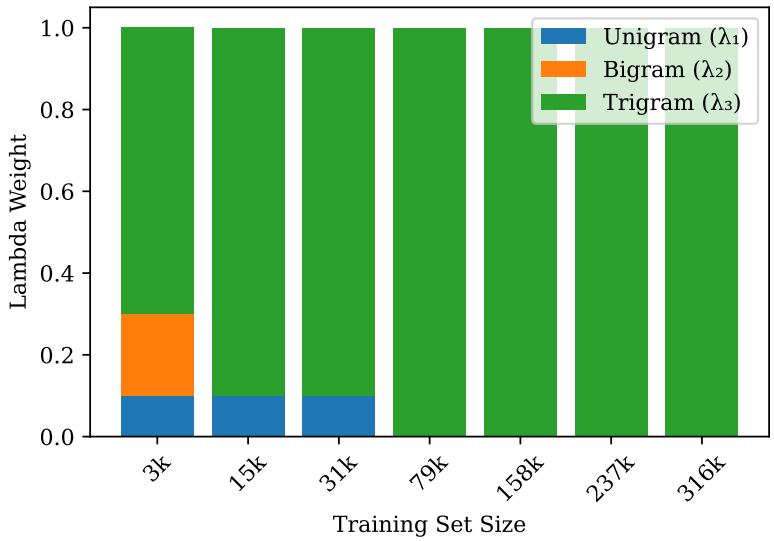

Figure 5: Lambda weight distribution as a function of training set size. With scarce data (3k words), the optimizer distributes weight across unigram, bigram, and trigram. As data increases, weight converges to pure trigram.

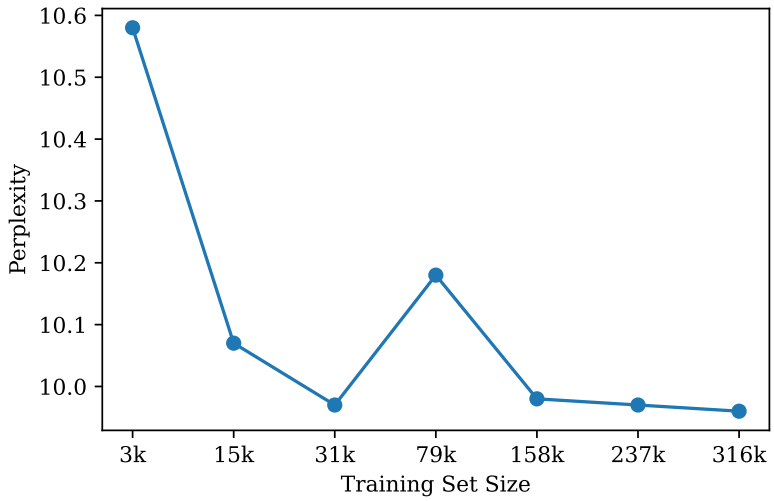

Figure 6: Perplexity decreases as training set size increases, with diminishing returns beyond approximately 30k words.

To test whether the lambda distribution changes with corpus size, we retrained the full pipeline on progressively smaller subsets of the training data, from 1% to 100%. Figure 5 shows the results. At 3,000 training words, the optimizer assigns weight to all three orders: 0.1 to the unigram, 0.2 to the bigram, and 0.7 to the trigram. As the corpus grows, the bigram weight drops to zero first, followed by the unigram, until the full corpus yields pure trigram weights. Figure 6 shows the corresponding perplexity, which decreases with training size and plateaus beyond approximately 30,000 words. Together, these plots demonstrate the adaptability of interpolation.

3.6 Word Length Analysis

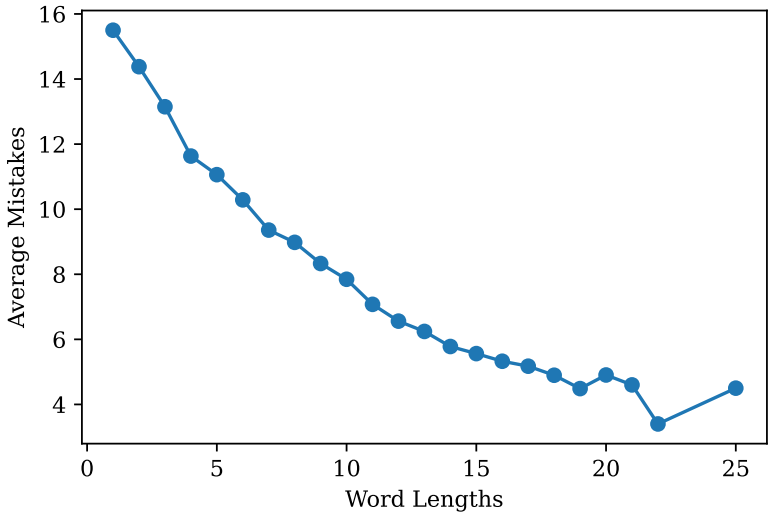

Figure 7: Average mistakes decrease with word length. Shorter words provide less context for the n-gram model, making prediction harder.

Figure 7 plots average mistakes as a function of word length across the full test set. Shorter words are substantially harder, they provide little context at the start of the game and each incorrect guess only eliminates one of 26 candidates. Longer words are more likely to contain common letter patterns that the model identifies early, and each correct guess reveals more positions, creating a feedback loop between revealed context and improved predictions. This also accounts for some of the noise in the perplexity–mistakes correlation (Figure 3).

3.7 Higher-Order N-grams

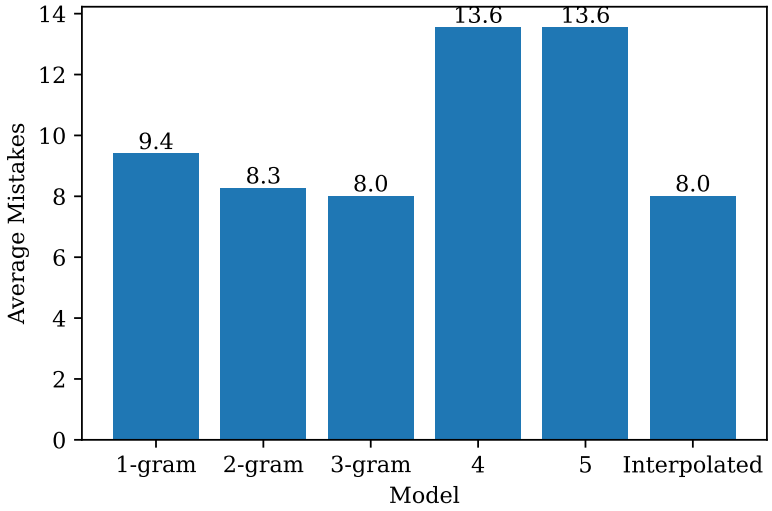

Figure 8: Extending to 4-gram and 5-gram models degrades performance due to data sparsity. The interpolated model assigns $\lambda_3 = 1.0$, correctly discarding the unreliable higher-order models.

Figure 8 extends the comparison to 4-gram and 5-gram models. Both perform substantially worse than the trigram in isolation, averaging 13.6 mistakes per word compared to the trigram's 8.0. Most 4- and 5-character contexts appear rarely or not at all in the training corpus, leaving the model with unreliable probability estimates. When all five orders are included in the interpolation, the optimizer assigns $\lambda_3 = 1.0$ and zeros out the 4-gram and 5-gram components entirely, correctly discarding the unreliable higher-order estimates. The grid search for five lambda weights required a coarser step size of 0.2 to remain computationally feasible, illustrating a scalability limitation of brute-force optimization as the number of mixture components grows.

3.8 Confusion Analysis

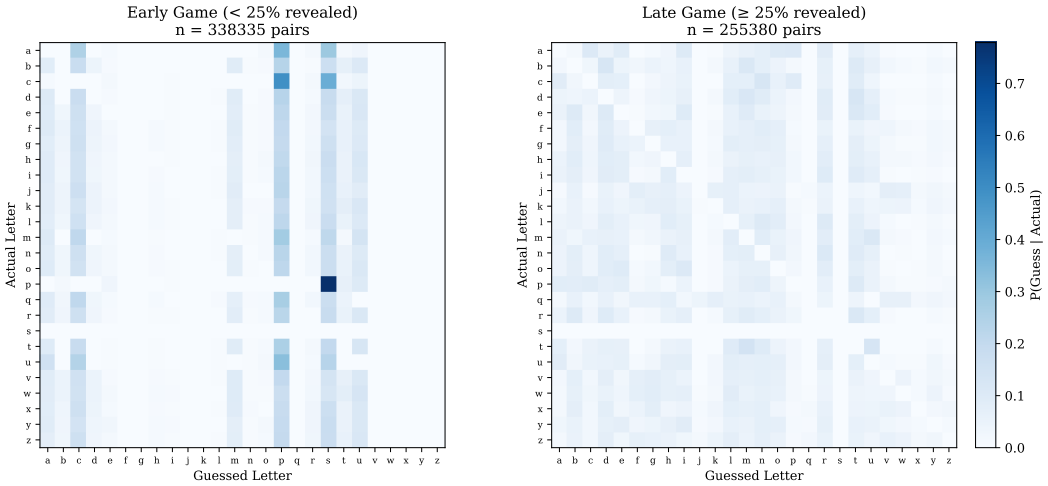

Figure 9: Confusion heatmaps for early-game ($< 25\%$ revealed) and late-game ($\geq 25\%$ revealed) guesses. Early-game errors are dominated by frequency-based guessing, while late-game errors show more context-dependent patterns.

To understand how error patterns change over the course of a game, we recorded every incorrect guess alongside the letters still needed at the time of the mistake. We split these pairs into early-game errors (fewer than 25% of positions revealed) and late-game errors (at or above that threshold). Figure 9 presents the resulting confusion heatmaps. The strong vertical banding in early-game heatmap indicates that, with little context revealed, the interpolated probabilities are dominated by the unigram distribution. Leading the model frequently guesses the same high-frequency letters regardless of what the word actually contains. The late-game heatmap shows a more diffuse pattern, indicating that the model leverages revealed context to make informed guesses.

4 Discussion and Future Work

A consistent theme across our experiments is that the interpolation framework self-regulates. When the corpus is large, the optimizer concentrates weight on the trigram and discards lower-order models (Section 3.4). When the corpus is small, it distributes weight across all orders (Section 3.5). When unreliable 4-gram and 5-gram components are added, it zeros them out (Section 3.7). In each case, the same perplexity-minimization procedure arrives at the appropriate model configuration without manual intervention. This adaptability is the primary practical advantage of interpolation over committing to a single n-gram order.

The experiments also reveal where the approach falls short. The model struggles most with short words (Section 3.6), where the lack of revealed context forces it to rely on marginal letter frequencies. The early-game confusion heatmap (Section 3.8) confirms that before enough letters are revealed, the model's guesses are frequency-driven. One direction for improvement is an adaptive strategy that adjusts the effective model order based on the current game state, leaning more heavily on unigram statistics early and shifting to higher-order context as letters are revealed.

A more fundamental limitation is the sparsity ceiling on model order. The 4-gram and 5-gram models perform worse than the trigram despite having access to more context, because most longer contexts are unattested in the training data. Character-level neural language models, such as the recurrent and convolutional architectures discussed in James et al. [2], address this by learning distributed representations of character contexts rather than storing explicit counts. A neural approach could in principle capture long-range dependencies, such as common suffixes like "-tion" or "-ness" influencing characters several positions earlier, without requiring every context to appear in the training corpus.

Finally, the grid search used for lambda optimization scales poorly. With three model orders, the search is fast; with five, we already needed a larger step size. Extending to more mixture components or finer resolution would require gradient-based or expectation-maximization approaches for weight estimation. Similarly, the solver treats each blank position independently when scoring candidate letters. A joint prediction model that considers the compatibility of letters across multiple positions simultaneously could reduce mistakes, particularly in the late game when only a few blanks remain and their fillers are mutually constrained.

5 Conclusion

The aim of this project was to apply the ideas behind large language models in a more tractable setting, by training and evaluating character-level n-gram models on the task of playing Hangman. A trigram model with linear interpolation smoothing achieves an average of 8.2 incorrect guesses per word on a 370,000-word English corpus. The interpolation weights, optimized by minimizing perplexity on held-out data, adapt automatically to corpus size by concentrating on the trigram when data is plentiful and distributing across all orders when data is scarce. Extending beyond the trigram to 4-gram and 5-gram models degrades performance, exposing the sparsity ceiling inherent to count-based language models. Perplexity, validated as a reliable predictor of per-word gameplay difficulty, proved to be an effective intrinsic metric for model selection in this task. The gap between what n-gram models can capture and what they cannot points naturally toward neural language models as a next step.

References

[1] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. 3rd edition draft, 2024.

[2] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning*. Springer, 2nd edition, 2021.

[3] dwyl. English words. `https://github.com/dwyl/english-words`, 2023. Accessed: 2026.

Appendix

A.1 Solver Entropy Analysis

To examine how the model's uncertainty evolves during a game, we computed the Shannon entropy of the solver's probability distribution at each guess. For each guess, the solver's score vector over candidate letters is normalized into a probability distribution, and its entropy $H = -\sum p \log_2 p$ is recorded alongside the fraction of the word revealed at that point and whether the guess was correct.

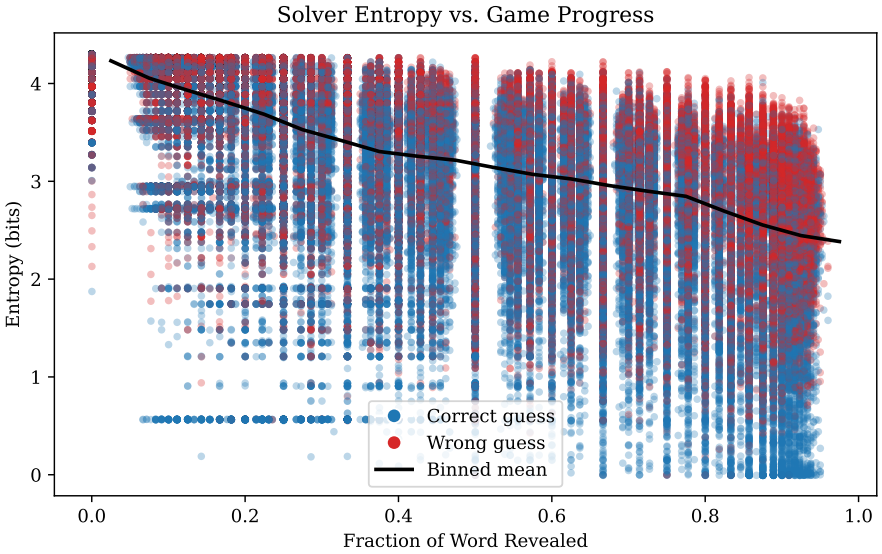

Figure 10: Solver entropy versus fraction of word revealed. Blue points are correct guesses, red are incorrect. The black line shows the binned mean. Entropy generally increases as the game progresses and common letters are exhausted.

Figure 10 plots entropy against the fraction of the word revealed, with correct guesses in blue and incorrect guesses in red. Across 290,104 total guesses (138,130 correct, 151,974 wrong), average entropy on correct guesses was 3.20 bits compared to 3.43 bits on incorrect guesses. The small gap suggests that the model's confidence level alone does not reliably distinguish correct from incorrect guesses.

A.2 Guess Accuracy by Game Progress

Figure 11 plots the fraction of correct guesses as a function of how much of the word has been revealed. The model substantially outperforms the random baseline at all stages, achieving approximately 42% accuracy with no letters revealed, peaking near 69% when roughly half the word is visible, then declining to around 35–40% in the late game as remaining letters tend to be rarer. Selected accuracy values by game progress are reported in Table 1.

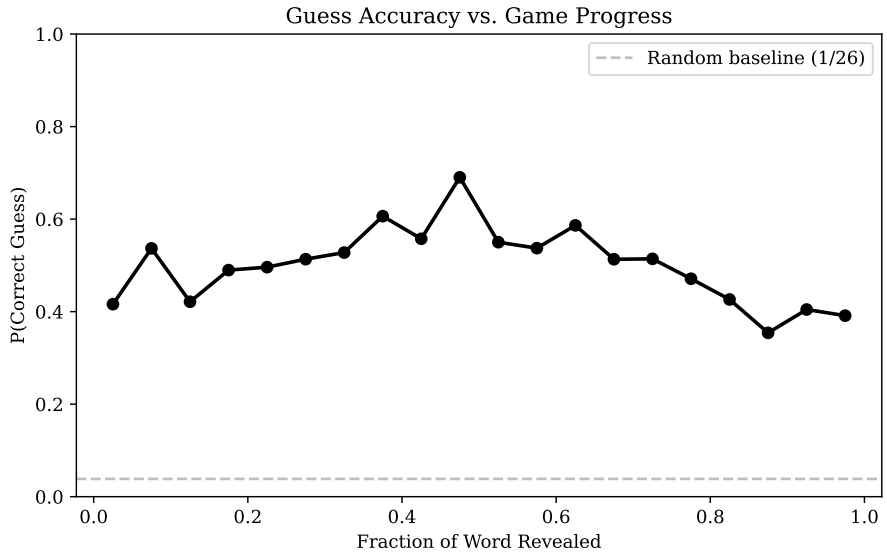

Figure 11: Guess accuracy (fraction correct) versus fraction of word revealed. The dashed line indicates the random baseline ($1/26 \approx 3.8\%$). Accuracy peaks near 50% revealed and declines as remaining letters become rarer.

Table 1: Guess accuracy at selected game stages, binned by fraction of word revealed.

% Revealed	Accuracy	n (guesses)
2%	41.6%	44,477
12%	42.1%	20,593
22%	49.6%	12,643
33%	52.8%	10,645
43%	55.7%	14,336
48%	69.0%	3,423
58%	53.7%	15,305
68%	51.3%	14,669
78%	47.1%	15,953
88%	35.4%	22,753

A.3 Training Size Sensitivity: Raw Data

Table 2 reports the exact lambda weights and perplexity values for each training subset used in Section 3.5. At the smallest training size (3,164 words), the optimizer assigns nonzero weight to all three n-gram orders. Beyond approximately 79,000 words, the trigram receives full weight and perplexity stabilizes near 9.97.

Table 2: Optimized lambda weights and test-set perplexity as a function of training set size.

Training Size	λ_1	λ_2	λ_3	Perplexity
3,164	0.1	0.2	0.7	10.58
15,821	0.1	0.0	0.9	10.07
31,643	0.1	0.0	0.9	9.97
79,109	0.0	0.0	1.0	10.18
158,219	0.0	0.0	1.0	9.98
237,329	0.0	0.0	1.0	9.97
316,439	0.0	0.0	1.0	9.96

A.4 Aggregate Confusion Heatmap

Figure 12 shows the confusion heatmap aggregated across all game stages, before the early/late split presented in the main paper (Figure 9). The vertical banding for high-frequency guesses is visible but attenuated compared to the early-game heatmap, since the aggregate mixes frequency-driven early guesses with context-driven late guesses. The cell values are partially influenced by the length distribution of words in the test set, not just the model's guessing behavior.

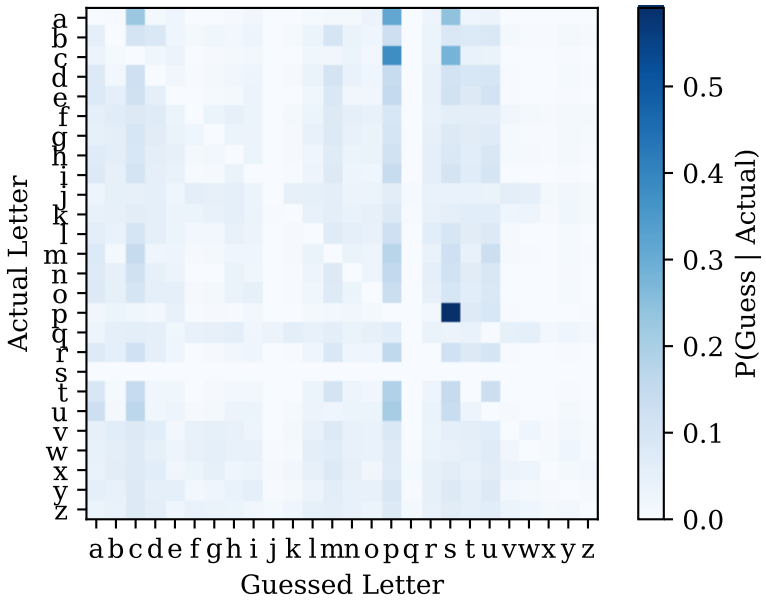

Figure 12: Aggregate confusion heatmap across all game stages. Vertical banding reflects the dominance of frequency-based guessing in the early game. Compare with the split heatmaps in Figure 9.

A.5 Source Code

The complete source code for the n-gram model, Hangman solver, and all experiments is available at:

`https://github.com/mindphil/mml/tree/main/hangman`

The repository contains two modules (`ngram_model.py` and `game.py`) and the experiment notebook used to generate all figures in this paper. A brief description of each module follows.

`ngram_model.py` Handles corpus loading, train/test splitting, n-gram extraction, model training via maximum likelihood estimation, linear interpolation of probabilities across model orders, perplexity computation, and grid-search optimization of interpolation weights.

`game.py` Implements the Hangman solver and gameplay logic. The solver pads the current game state with boundary tokens, computes interpolated probabilities for each candidate letter at each blank position, sums scores across positions, and returns the highest-scoring letter. The module also provides functions for playing complete games and evaluating average mistake counts over a test set.